Set Interface example programs

Set Interface and Its Implementations in Java Collections Framework

1. The Set Interface

Set<E> (in java.util) extends Collection<E> and models the mathematical set:

No duplicates: add() returns false if the element is already present.

At most one null (implementation-dependent; TreeSet rejects it).

No index-based access: there is no get(int), unlike List.

Duplicate detection relies on equals() (and hashCode() for hash-based sets, or compareTo()/Comparator for TreeSet).

Figure:Set Hierarchy

Methods of Set Interface

HashSet

Backed by a HashMap. Each element you add becomes a key in the internal map, with a dummy constant object as the value. The element's hashCode() selects a bucket, and equals() resolves collisions within the bucket (linked list, converted to a balanced tree when a bucket grows past 8 entries in Java 8+).

Characteristics:

No ordering guarantee, and the order can change as the set resizes

Allows one null

add, remove, contains are O(1) on average

Not synchronized

Default capacity 16, load factor 0.75

```
import java.util.*;
public class HashSetDemo {
public static void main(String[] args) {
```

Set<String> courses = new HashSet<>();

```
System.out.println(courses.add("DBMS")); // true
```

courses.add("Java");

courses.add("C++");

courses.add("Networks");

```
System.out.println(courses.add("Java")); // false (duplicate)
```

courses.add(null); // allowed (one null)

```
System.out.println(courses); // order unpredictable
System.out.println(courses.contains("DBMS")); // true
```

courses.remove("C++");

```
System.out.println("Size: " + courses.size());
```

```
for (String c : courses) {
System.out.println(c);
}
}
}
```

Important:

custom objects need equals() and hashCode()

```
class Student {
int id;
String name;
Student(int id, String name) { this.id = id; this.name = name; }
@Override
public boolean equals(Object o) {
if (this == o) return true;
if (!(o instanceof Student)) return false;
Student s = (Student) o;
return id == s.id && name.equals(s.name);
}
@Override
public int hashCode() {
return Objects.hash(id, name);
}
@Override
public String toString() { return id + ":" + name; }
}
public class CustomHashSet {
public static void main(String[] args) {
Set<Student> set = new HashSet<>();
set.add(new Student(1, "Arun"));
set.add(new Student(1, "Arun")); // duplicate, rejected
set.add(new Student(2, "Meena"));
System.out.println(set.size()); // 2
}
}
```

3. LinkedHashSet

Extends HashSet, but backed by a LinkedHashMap. It keeps the hash table for fast lookup and adds a doubly linked list running through all entries to record insertion order.

Characteristics:

Predictable iteration order = insertion order

Re-inserting an existing element does not change its position

Allows one null

O(1) for basic operations, slightly slower and more memory-hungry than HashSet

```
import java.util.*;
public class LinkedHashSetDemo {
public static void main(String[] args) {
```

Set<String> lhs = new LinkedHashSet<>();

lhs.add("Banana");

lhs.add("Apple");

lhs.add("Mango");

lhs.add("Apple"); // ignored, order unchanged

lhs.add("Cherry");

```
System.out.println(lhs); // [Banana, Apple, Mango, Cherry]
```

}

}

use case: removing duplicates while preserving order

List<Integer> list = Arrays.asList(5, 3, 5, 1, 3, 9, 1);

Set<Integer> unique = new LinkedHashSet<>(list);

```
System.out.println(unique); // [5, 3, 1, 9]
```

4. TreeSet

Internal working: Backed by a TreeMap, which is a Red-Black tree (a self-balancing binary search tree). Elements are kept sorted at all times.

Characteristics:

Sorted in natural order (Comparable) or by a supplied Comparator

add, remove, contains are O(log n)

Does not allow null (throws NullPointerException, since comparison is needed)

Elements must be mutually comparable, otherwise ClassCastException

Duplicates are determined by compareTo()/compare() returning 0, not by equals()

Implements NavigableSet, giving range and neighbor operations

```
import java.util.*;
public class TreeSetDemo {
public static void main(String[] args) {
```

TreeSet<Integer> ts = new TreeSet<>();

ts.add(40); ts.add(10); ts.add(30); ts.add(20); ts.add(10);

```
System.out.println(ts); // [10, 20, 30, 40]
```

```
System.out.println(ts.first()); // 10
System.out.println(ts.last()); // 40
System.out.println(ts.lower(30)); // 20 (strictly less)
System.out.println(ts.floor(30)); // 30 (less or equal)
System.out.println(ts.higher(30)); // 40 (strictly greater)
System.out.println(ts.ceiling(25)); // 30 (greater or equal)
System.out.println(ts.headSet(30)); // [10, 20]
System.out.println(ts.tailSet(30)); // [30, 40]
System.out.println(ts.subSet(10, 40)); // [10, 20, 30]
System.out.println(ts.descendingSet()); // [40, 30, 20, 10]
System.out.println(ts.pollFirst()); // 10 (removes it)
```

}

}

Custom sorting with Comparable

class Employee implements Comparable<Employee> {

int id; String name; double salary;

Employee(int id, String name, double salary) {

this.id = id; this.name = name; this.salary = salary;

}

@Override

public int compareTo(Employee other) {

```
return Integer.compare(this.id, other.id); // natural order: by id
```

}

@Override

```
public String toString() { return id + " " + name + " " + salary; }
```

}

```
public class TreeSetComparable {
public static void main(String[] args) {
```

Set<Employee> set = new TreeSet<>();

set.add(new Employee(103, "Ravi", 52000));

set.add(new Employee(101, "Kavya", 61000));

set.add(new Employee(102, "Divya", 58000));

set.forEach(System.out::println); // sorted by id

}

}

Custom sorting with Comparator

Set<Employee> bySalaryDesc = new TreeSet<>(

Comparator.comparingDouble((Employee e) -> e.salary).reversed()

);

bySalaryDesc.add(new Employee(103, "Ravi", 52000));

bySalaryDesc.add(new Employee(101, "Kavya", 61000));

bySalaryDesc.add(new Employee(102, "Divya", 58000));

bySalaryDesc.forEach(System.out::println); // highest salary first

If two different employees have the same salary and the comparator compares only salary, the second is treated as a duplicate and silently dropped. Add a tie-breaker:

Comparator.comparingDouble((Employee e) -> e.salary).reversed()

.thenComparing(e -> e.id);

HashSet: fastest; use when you only need uniqueness and fast membership tests.

LinkedHashSet: use when you need uniqueness and want to preserve insertion order (e.g., de-duplicating user input, LRU-style structures).

TreeSet: use when you need elements always sorted, or range/neighbor queries (floor, ceiling, subSet).

7. Set Operations (Mathematical)

Set<Integer> a = new TreeSet<>(Arrays.asList(1, 2, 3, 4, 5));

Set<Integer> b = new TreeSet<>(Arrays.asList(4, 5, 6, 7));

Set<Integer> union = new TreeSet<>(a);

union.addAll(b); // [1, 2, 3, 4, 5, 6, 7]

Set<Integer> intersection = new TreeSet<>(a);

intersection.retainAll(b); // [4, 5]

Set<Integer> difference = new TreeSet<>(a);

difference.removeAll(b); // [1, 2, 3]

boolean subset = a.containsAll(intersection); // true

Method	Description
add(element)	Adds element if not already present. Returns true if added.
addAll(collection)	Adds all elements from the given collection.
clear()	Removes all elements from the set.
contains(element)	Checks if the set contains the specified element.
containsAll(collection)	Checks if the set contains all elements from the given collection.
hashCode()	Returns the hash code of the set.
isEmpty()	This method is used to check whether the set is empty or not.

iterator()	This method is used to return the iterator of the set.
remove(element)	Removes the specified element from the set.
removeAll(collection)	Removes all elements in the given collection from the set.
retainAll(collection)	Retains only elements present in the given collection.
size()	Returns the number of elements in the set.
toArray()	This method is used to form an array of the same elements as that of the Set.

Feature	HashSet	LinkedHashSet	TreeSet
Underlying structure	Hash table (HashMap)	Hash table + doubly linked list (LinkedHashMap)	Red-Black tree (TreeMap)
Ordering	None	Insertion order	Sorted (natural/comparator)
Time complexity (add/remove/contains)	O(1) avg	O(1) avg	O(log n)
Null elements	One allowed	One allowed	Not allowed
Duplicate check	hashCode() + equals()	hashCode() + equals()	compareTo() / compare()
Memory overhead	Lowest	Higher	Higher (tree nodes)
Extra interfaces	none	none	SortedSet, NavigableSet
Thread-safe	No	No	No